

22. 括号生成

Category	Difficulty	Likes	Dislikes	ContestSlug	ProblemIndex	Score
algorithms	Medium	(Not Available)	(Not Available)	-	-	0

- **Tags:** 字符串, 动态规划, 回溯
- **Companies:** (Not Available)

数字 n 代表生成括号的对数，请你设计一个函数，用于能够生成所有可能的并且有效的括号组合。

示例 1:

输入: $n = 3$

输出: ["((()))","(()())","(())()","()(())","()()()"]

示例 2:

输入: $n = 1$

输出: ["()"]

提示:

- $1 \leq n \leq 8$

Discussion | Solution

解决方法

1. 问题理解

目标: 生成 n 对括号的所有可能的、有效的组合。有效括号组合的定义: 1. 左括号和右括号的数量必须相等 (都为 n)。2. 在任何位置, 已经出现的左括号数量必须大于或等于已经出现的右括号数量。

输入: 整数 n (括号对数)。输出: 包含所有有效括号组合的字符串列表。

2. 思路分析

这个问题非常适合使用回溯算法。我们需要在构建字符串的过程中, 不断做出添加左括号 '(' 或右括号 ')' 的决策, 并确保每一步都满足有效括号的约束条件。

决策树模型: - 树的每个节点代表一个部分构建的括号字符串。- 从一个节点出发, 可以尝试添加 '(' 或 ')'。- 路径的有效性需要实时检查。

回溯过程:

1. 状态/路径 (**State/Path**):
 - 当前构建的括号字符串 `current_string`。
 - 已使用的左括号数量 `open_count`。
 - 已使用的右括号数量 `close_count`。
2. 选择列表 (**Choices**): 在当前状态下, 可以添加 '(' 或 ')'。
3. 约束条件/剪枝 (**Constraints/Pruning**):
 - 可以添加左括号 '(' 的条件: 只要已使用的左括号数量 `open_count` 小于 `n` (`open_count < n`)。
 - 可以添加右括号 ')' 的条件: 只有当已使用的右括号数量 `close_count` 小于已使用的左括号数量 `open_count` (`close_count < open_count`) 时, 才能添加右括号。这个条件保证了括号组合的有效性 (不会出现) (这种非法前缀)。
4. 结束条件 (**End Condition**): 当构建的字符串 `current_string` 的长度达到 $2 * n$ 时 (即使用了 `n` 个左括号和 `n` 个右括号), 说明找到了一个完整的、有效的括号组合。

具体步骤:

1. 定义回溯函数 `backtrack(open_count, close_count, current_string)`:
 - `open_count`: 已使用的左括号数。
 - `close_count`: 已使用的右括号数。
 - `current_string`: 当前构建的字符串。
2. 递归终止条件:
 - 如果 `len(current_string) == 2 * n`:
 - 将 `current_string` 加入结果列表 `result`。
 - 返回。
3. 递归步骤 (尝试添加括号):
 - 尝试添加左括号 '(':
 - 如果 `open_count < n`:
 - * 调用 `backtrack(open_count + 1, close_count, current_string + '(')`。
 - 尝试添加右括号 ')':
 - 如果 `close_count < open_count`:
 - * 调用 `backtrack(open_count, close_count + 1, current_string + ')')`。
4. 主函数调用:
 - 初始化结果列表 `result = []`。
 - 调用 `backtrack(0, 0, "")` 开始回溯。
 - 返回 `result`。

3. 代码实现 (Python)

```
from typing import List

class Solution:
    def generateParenthesis(self, n: int) -> List[str]:
```

```

"""
使用回溯算法生成所有有效的括号组合。
Args:
    n: 括号的对数。
Returns:
    包含所有有效括号组合的字符串列表。
"""
result = [] # 存储最终结果

def backtrack(open_count: int, close_count: int, current_string: str):
    """
    回溯函数核心逻辑。
    Args:
        open_count: 已使用的左括号数量。
        close_count: 已使用的右括号数量。
        current_string: 当前构建的括号字符串。
    """
    # 递归终止条件: 当字符串长度达到 2*n 时, 找到一个完整组合
    if len(current_string) == 2 * n:
        result.append(current_string)
        return

    # 尝试添加左括号 '('
    # 条件: 已使用的左括号数量小于 n
    if open_count < n:
        # 做选择 (隐式), 递归
        backtrack(open_count + 1, close_count, current_string + '(')
        # 撤销选择 (隐式)

    # 尝试添加右括号 ')'
    # 条件: 已使用的右括号数量小于已使用的左括号数量 (保证有效性)
    if close_count < open_count:
        # 做选择 (隐式), 递归
        backtrack(open_count, close_count + 1, current_string + ')')
        # 撤销选择 (隐式)

    # 初始状态: 0 个左括号, 0 个右括号, 空字符串
    backtrack(0, 0, "")
    return result

# 示例调用
# sol = Solution()
# print(sol.generateParenthesis(3))
# print(sol.generateParenthesis(1))

```

4. 复杂度分析

- 时间复杂度: $O(C_n)$ 或 $O(4^n / \sqrt{n})$, 其中 C_n 是第 n 个卡特兰数 (Catalan Number).
 - 生成的有效括号组合的数量是卡特兰数 $C_n = (1/(n+1)) * C(2n, n)$, 大约是 $O(4^n / (n^{3/2}))$ 。
 - 回溯算法会探索所有有效的路径, 每个有效组合的生成可以看作是常数时间的操作 (如果忽略字符串拼接成本) 或者 $O(N)$ (如果考虑字符串拼接成本)。
 - 状态树的节点数与卡特兰数相关。总的时间复杂度与最终解的数量级相关。
- 空间复杂度: $O(N)$
 - 递归调用栈的深度最多为 $2N$ 。
 - `current_string` 在递归过程中最大长度为 $2N$ 。
 - 结果列表 `result` 的空间取决于解的数量 C_n 和解的长度 $2N$, 为 $O(N * C_n)$, 通常不计入算法本身空间复杂度。

5. 如何在面试中讲解

1. 问题理解:
 - “目标是生成 n 对括号的所有可能的、有效的组合。”
 - “有效性意味着: 左右括号数量相等, 且任何前缀中左括号数量不少于右括号数量。”
2. 核心思路: 回溯:
 - “这是一个构建序列的问题, 并且需要在构建过程中满足约束条件, 非常适合回溯。”
 - “我们可以一步步构建括号字符串, 每次决定是添加 ‘(’ 还是 ‘)’。”
3. 回溯状态与约束:
 - “定义回溯函数 `backtrack(openCount, closeCount, currentString)`。”
 - “状态: `openCount` 是已用左括号数, `closeCount` 是已用右括号数, `currentString` 是当前字符串。”
 - “约束/剪枝:”
 - “添加 ‘(’ 的条件是 `openCount < n`。”
 - “添加 ‘)’ 的条件是 `closeCount < openCount`。这个条件是保证括号有效的关键, 确保不会在不应该出现右括号的地方添加它。”
4. 回溯过程:
 - “结束条件: 当 `len(currentString) == 2 * n` 时, 表示找到了一个完整的有效组合, 加入结果列表。”
 - “递归步骤:”
 - “如果满足添加 ‘(’ 的条件, 则递归调用 `backtrack(openCount + 1, closeCount, currentString + '(')`。”
 - “如果满足添加 ‘)’ 的条件, 则递归调用 `backtrack(openCount, closeCount + 1, currentString + ')')`。”
 - “由于字符串不可变, 状态恢复是自动的。”
5. 初始调用:
 - “从 `backtrack(0, 0, "")` 开始。”

6. 复杂度分析:

- “最终结果的数量是第 n 个卡特兰数 C_n 。”
- “时间复杂度与结果数量相关，大致是 $O(C_n)$ 或 $O(4^n / \sqrt{n})$ 。”
- “空间复杂度是 $O(N)$ ，主要是递归深度。”